



聊聊volatile

北京理工大学计算机学院
金旭亮

概述



`volatile`字面有“易挥发”、“不稳定”的含义，引申到软件领域，就是“变量值易丢失”的意思。



`volatile`主要用于修饰**多线程共享的变量**，表示此变量的值容易变化（即被其他线程更改），因而“**不稳定**”。



如果一个字段被声明成`volatile`，Java内存模型（JMM）会确保所有线程看到这个变量的值都是一致的，这主要是通过强制所有线程都从主内存中读取变量值来实现的。

关于volatile关键字



volatile在使用上具有以下技术特性：

- ▶ 保障可见性： B线程能马上看到A线程更改的数据。
- ▶ 保障原子性： 在X86架构64位JDK版本中，可以让写double或long值这样的操作变成原子的。
- ▶ 可以禁止代码重排序。

使用volatile定义变量，Java编译器会生成相应指令，强制线程每次从主内存中去读取变量，而不是从私有内存中去读取，这样就保证了线程之间数据的可见性。

volatile技术内幕



volatile内部的工作原理相当复杂，它涉及到多核CPU所使用之高速缓存的数据同步协议，而且不同的CPU、高速缓存、总线和内存之间的数据同步方式还不一样。



volatile还涉及到特定平台JVM的实现以及Java编译器的代码优化，它会对“指令重排”这一硬件和软件优化手段产生影响。



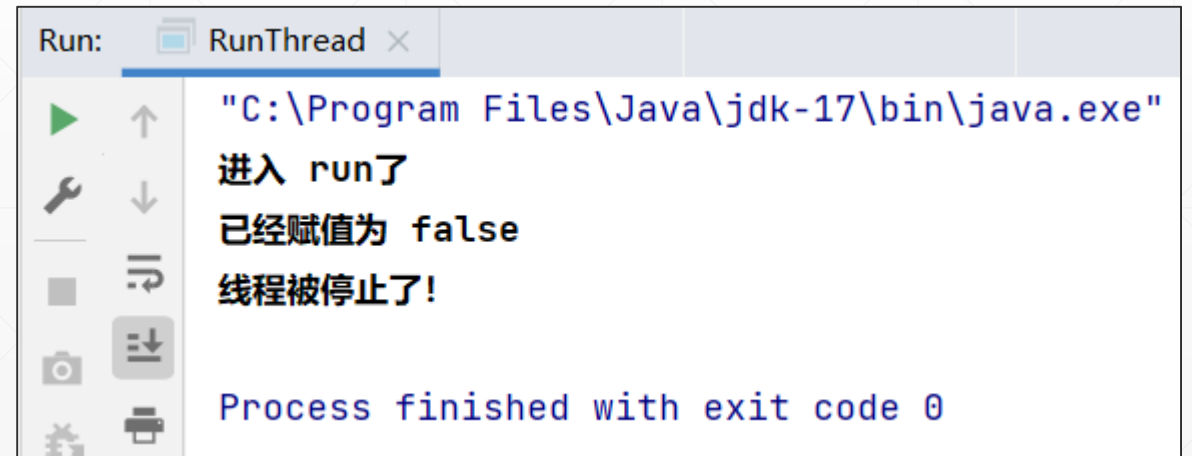
鉴于其复杂性和涉及面过广，本课程只介绍其基本用法，而不作过多介绍，对此感兴趣的同学请自行收集和阅读相关的专业技术资源进行深入钻研。

volatile导致死循环

示例中的线程类定义了一个isRunning字段，它的值是run方法中循环的判断条件。

isRunning字段必须添加volatile关键字，否则，线程将陷入死循环。

```
public class UseVolatileThreadDemo extends Thread {  
    //不加volatile, 将会死循环  
    3 usages  
    private volatile boolean isRunning = true;  
    public boolean isRunning() {  
        return isRunning;  
    }  
    1 usage  
    public void setRunning(boolean isRunning) {  
        this.isRunning = isRunning;  
    }  
    @Override  
    public void run() {  
        System.out.println("进入run了");  
        while (isRunning == true) {  
        }  
        System.out.println("线程被停止了!");  
    }  
  
    public static void main(String[] args)  
        throws InterruptedException {...}  
}
```



```
Run: RunThread x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
进入 run了  
已经赋值为 false  
线程被停止了!  
Process finished with exit code 0
```

基于volatile实现Singleton模式

```
class SingletonUseDCL {  
    4 usages  
    private static volatile SingletonUseDCL instance;  
    1 usage  
    private SingletonUseDCL() {  
    }  
    2 usages  
    public static SingletonUseDCL getInstance() {  
        if (null == instance) {  
            synchronized (SingletonUseDCL.class) {  
                if (null == instance) {  
                    instance = new SingletonUseDCL();  
                }  
            }  
        }  
        return instance;  
    }  
}
```

左图所示为代码为著名的
(Double-checked Locking,
DCL) 模式，其要点为：

- (1) 构造方法私有
- (2) 私有静态volatile字段
- (3) 两次if条件判断
- (4) 锁定Class对象

双重检查锁定这种方法目前已经被
视为“反模式 (Anti-Pattern)”，
即不再提倡使用的方法。

volatile vs. synchronized



关键字volatile和synchronized的使用场景总结如下：



volatile主要用于实现多线程共享变量的最新值。



synchronized主要用于定义临界区，实现临界区内代码的多线程互斥访问。

volatile是一种“轻量级”的锁



在特定场景下，可使用volatile变量替代锁。



可使用volatile保障多线程共享变量的可见性。



可使用volatile变量作为状态标志。